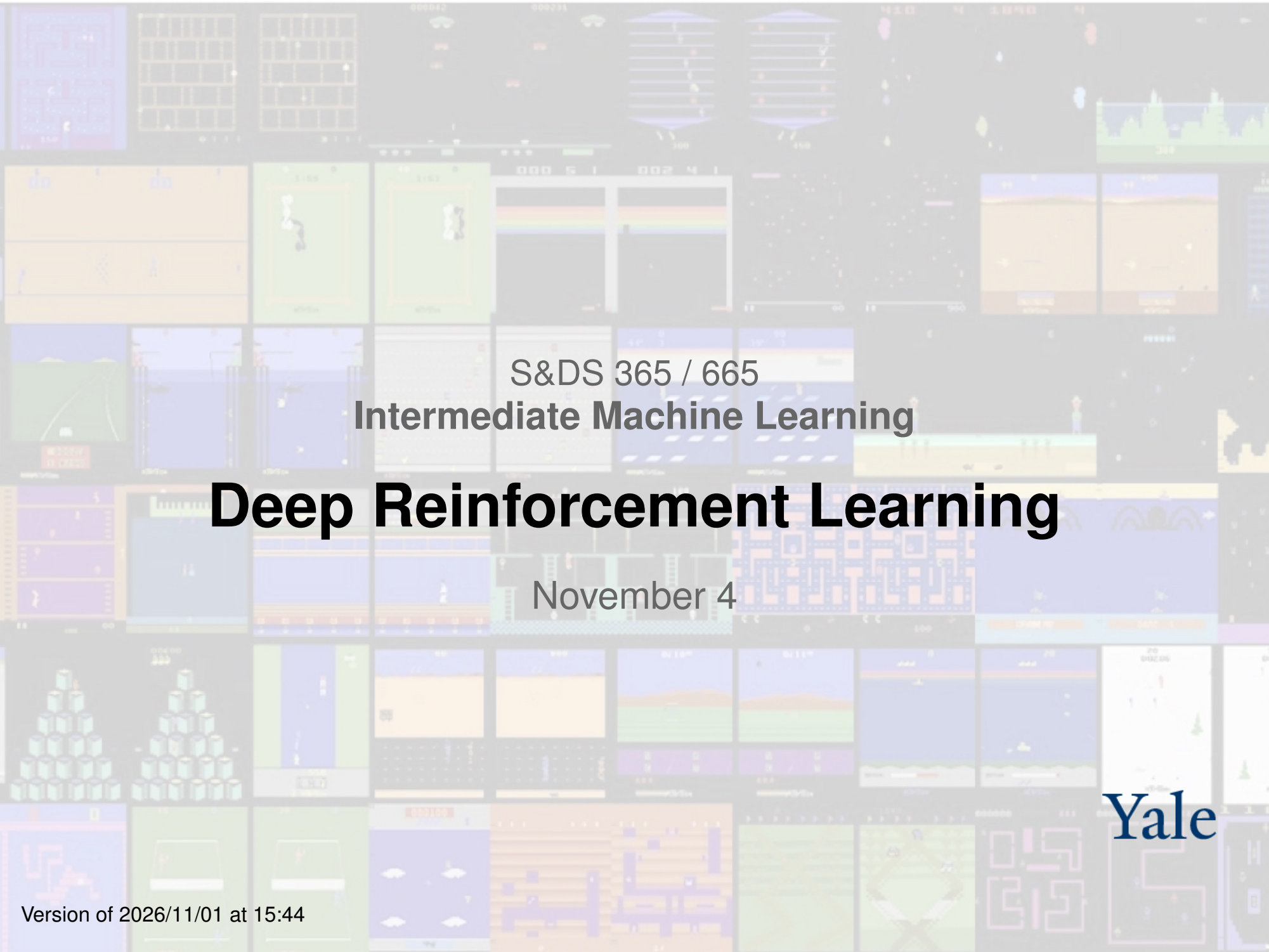


S&DS 365 / 665

Intermediate Machine Learning

Deep Reinforcement Learning

November 4

Yale

Version of 2026/11/01 at 15:44

Reminders

- Assignment 3 due tomorrow
- Assignment 4 out today (GNNs and Deep RL)
- Quiz 4 next week (Wednesday, in class)
- Project proposals due next week (Thursday)
- Exam 1 sample solutions posted to Canvas

Outline

- Quick recap
  - *Q*-learning
  - Illustration on taxi problem
  - RL concepts
- Bellman equation
- Deep reinforcement learning
- Policy gradient methods

Main reference: “Reinforcement Learning: An Introduction” (Second Edition), Richard S. Sutton and Andrew G. Barto

Reinforcement learning

- An agent interacts with an environment
- The actions the agent takes change the state of the environment
- The agent receives rewards for each action, and seeks to maximize the total cumulative reward

Reinforcement learning is a framework for sequential decision making to achieve a long-term goal.

Reinforcement learning: Formalization

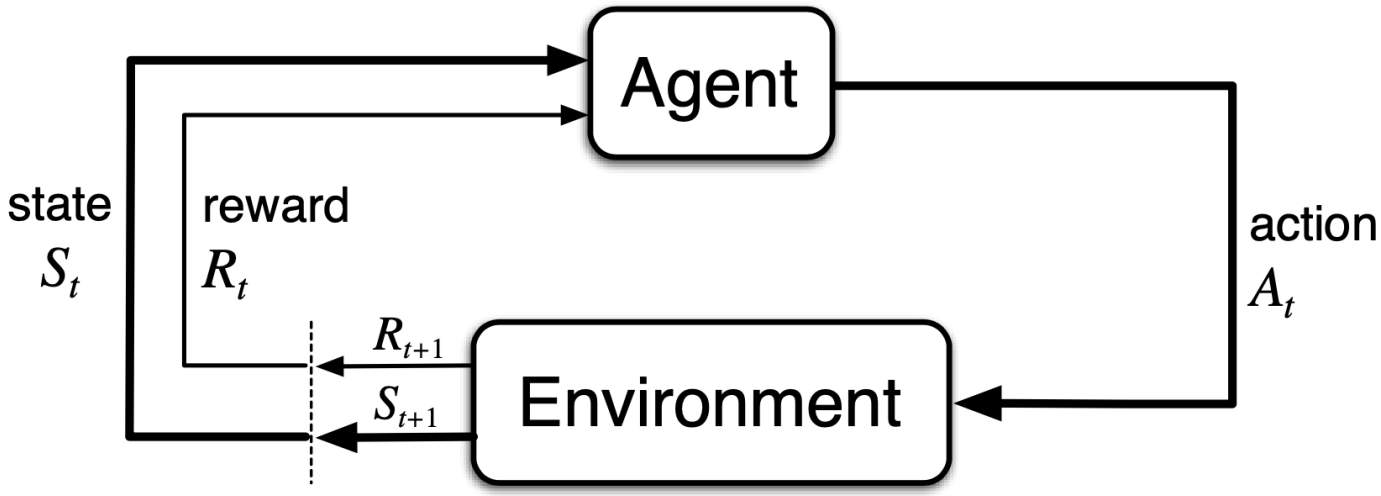
- The environment is in state s at a given time
- The agent takes action a
- The environment transitions to state $s' = \text{next}(s, a)$
- The agent receives reward $r = \text{reward}(s, a)$

Reinforcement learning: Formalization

- The environment is in state s at a given time
- The agent takes action a
- The environment transitions to state $s' = \text{next}(s, a)$
- The agent receives reward $r = \text{reward}(s, a)$

This is said to be a *Markov decision process*. It's "Markov" because the next state only depends on the current state and the action selected. It's a "decision process" because the agent is making choices of actions in a sequential manner.

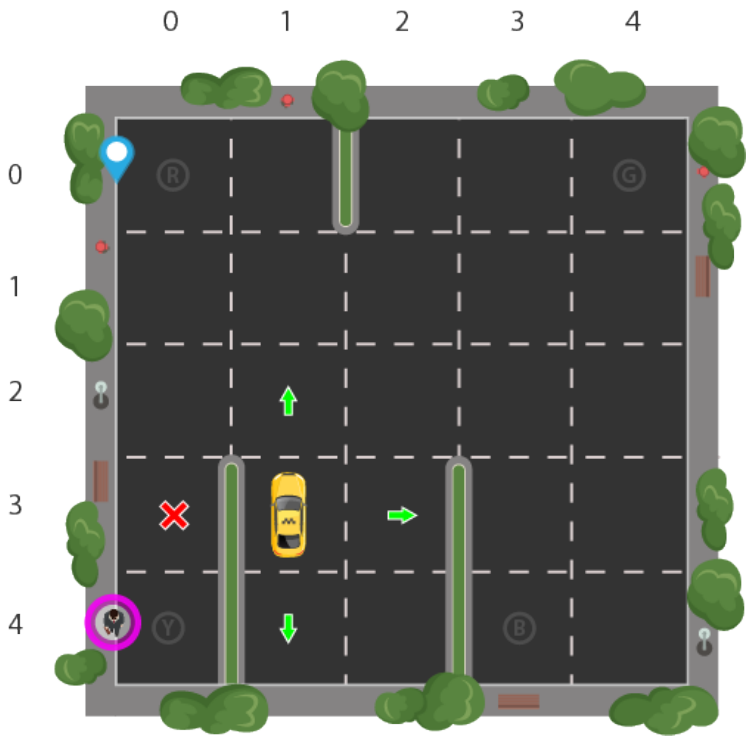
Keep in mind the basic setup



"Reinforcement Learning: An Introduction" (Second Edition), Richard S. Sutton and Andrew G. Barto

Recall: Taxi problem

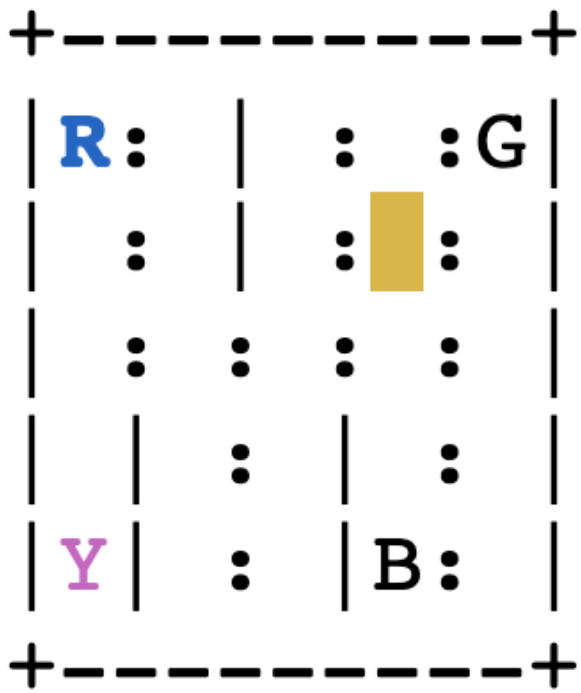
A taxicab drives around the environment, picking up and delivering a passenger at four locations



Recall: Taxi problem

A taxicab drives around the environment, picking up and delivering a passenger at four locations

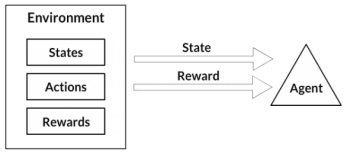
”Ascii art” rendition:



+ Code + Text

Reinforcement Learning

In reinforcement learning, an agent interacts with the environment, experiencing a series of rewards based on its actions. The agent seeks to maximize its rewards by developing a strategy that learns to choose appropriate actions in each state.

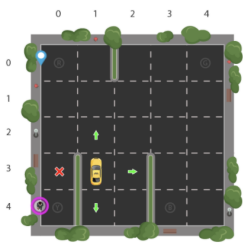


In this notebook we demo the Q-learning algorithm, one of the fundamental algorithms of reinforcement learning. We illustrate Q-learning on the taxicab problem formulated by Tom Dietterich in the paper "Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition", as developed in the code from OpenAI gym. Our presentation follows this tutorial.

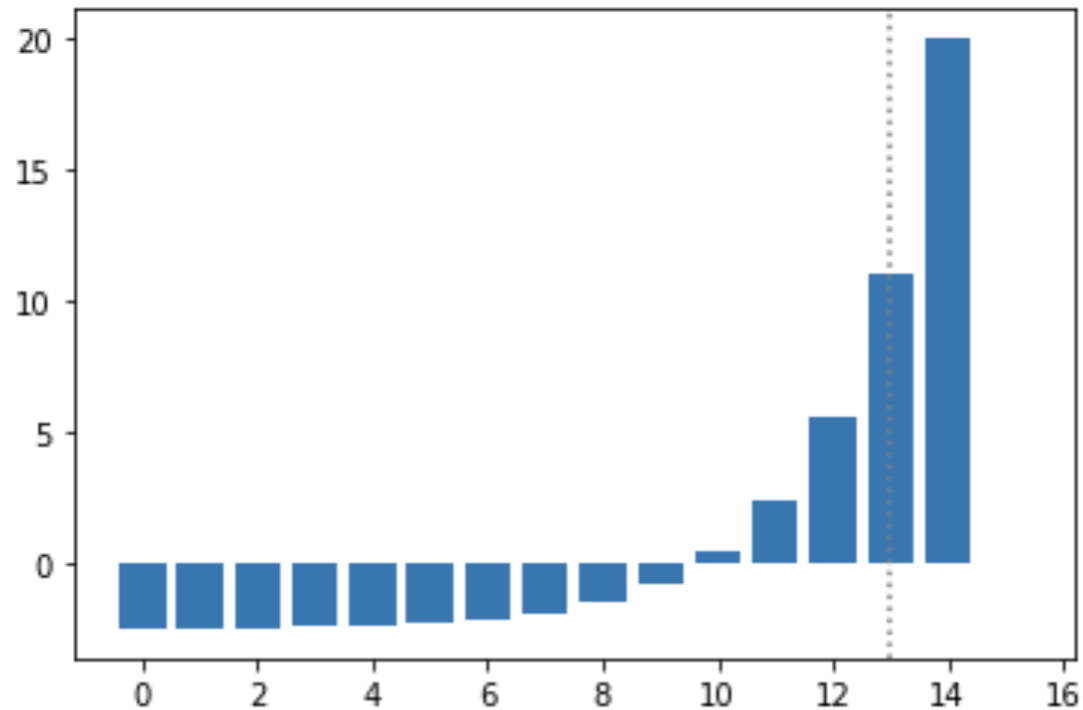
We'll need the OpenAI gym package. This can be installed as shown below. We'll display some simple graphics using `IPython.display`.

```
[ ]  #!pip install gym
     import gym
     import numpy as np
     from IPython.display import clear_output
     from time import sleep
```

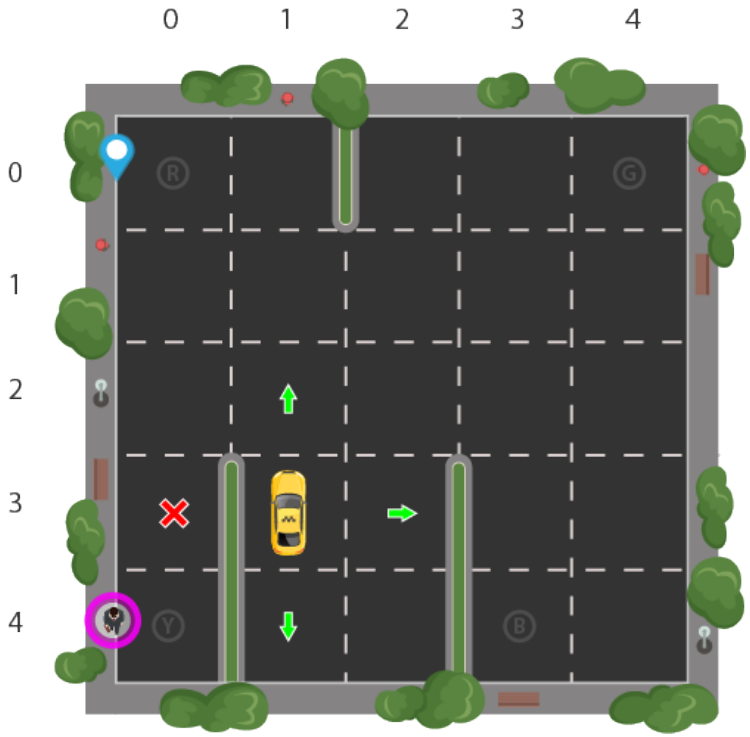
The environment is a simple grid, with some barriers inserted to make things more interesting. A taxicab drives around the environment, picking up and delivering a passenger at four locations. A graphic of the environment is shown below.



```
+---------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+---------+
  (East)
```



No map? No problem!



In Q-learning, the agent learns to navigate optimally without learning an explicit map.

Proof of convergence: Watkins and Dayan, “Q-Learning,” *Machine Learning*, 1992

Bellman equation



The optimal value function is the largest expected discounted long term reward starting from that state.

---

https://en.wikipedia.org/wiki/Richard_E._Bellman

Bellman equation: Deterministic case

The optimality condition for the value function v_* is

$$v_*(s) = \max_a \Big\{ \text{reward}(s, a) + \gamma v_*(\text{next}(s, a)) \Big\}$$

Bellman equation: Deterministic case

The optimality condition for the Q-function is

$$Q_*(s, a) = \text{reward}(s, a) + \gamma \max_{a'} Q_*(\text{next}(s, a), a')$$

and then $v_*(s) = \max_{a'} Q_*(s, a')$

Q-learning update

Note how this makes sense in terms of the update rule:

$$Q(s,a) \longleftarrow Q(s,a) + \alpha\Big(\text{reward}(s,a) + \gamma \max_{a'} Q(\text{next}(s,a), a') - Q(s,a)\Big)$$

Bellman equation: Deterministic case

If we know Q_*, we know v_*:

Bellman equation: Deterministic case

If we know Q_*, we know v_*:

$$
\begin{aligned}
v_*(s) &= \max_a Q_*(s,a) \\
&= \max_a \left\{ \text{reward}(s,a) + \gamma \max_{a'} Q_*(\text{next}(s,a), a') \right\} \\
&= \max_a \left\{ \text{reward}(s,a) + \gamma v_*(\text{next}(s,a)) \right\}
\end{aligned}
$$

which is the Bellman equation

Bellman equation: Random environments

The optimality condition for the value function v_* is

$$\begin{aligned} v_*(s) &= \max_a \sum_{s',r} p(s', r \mid s, a)\Big\{r + \gamma v_*(s')\Big\} \\ &= \max_a \mathbb{E}\Big[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a\Big] \end{aligned}$$

Bellman equation: Random environments

Value function optimality

$$v_*(s) = \max_a \mathbb{E}\Big[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a\Big]$$

Bellman equation: Random environments

The optimality condition for the Q-function

$$
\begin{aligned}
Q_*(s,a) &= \sum_{s',r} p(s',r \mid s,a)\Big\{ r + \gamma \max_{a'} Q_*(s',a') \Big\} \\
&= \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} Q_*(S_{t+1},a') \;\mid\; S_t = s, A_t = a \Big]
\end{aligned}
$$

Bellman equation: Random environments

Q-function optimality

$$Q_*(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} Q_*(S_{t+1}, a') \mid S_t = s, A_t = a\Big]$$

Comment on Q-learning

- Q-learning is an example of *temporal difference (TD) learning*
- It is an "off-policy" approach that is practical if the space of actions is small
- Value iteration is analogous approach for learning the value function for a given policy π, a (possibly random) choice of action for each state

Deep reinforcement learning: Motivation

- Direct implementation of Q-learning only possible for small state and action spaces
- For large state spaces we need to map states to “features”
- Deep RL uses a multilayer neural network to learn these features and the Q-function

Starting point: Bellman equation

$$Q(s, a; \theta) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta) \mid S_t = s, A_t = a\right]$$

- The parameters θ are weights in a neural network
- The state S_{t+1} is the input to the network
- Each possible action a is assigned a value by the network

How do we solve this implicit equation for the network parameters?

Strategy

Objective:

$$Q(s, a; \theta) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta) \mid S_t = s, A_t = a\Big]$$

Let y_t be a sample from this conditional distribution:

$$y_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta_{\text{current}})$$

Adjust the parameters θ to make the squared error small (SGD):

$$(y_t - Q(s, a; \theta))^2$$

Strategy

Adjust the parameters θ to make the squared error small

$$(y_t - Q(s, a; \theta))^2$$

How? Carry out SGD

$$\theta \longleftarrow \theta + \eta \left(y_t - Q(s, a; \theta)\right) \nabla_\theta Q(s, a; \theta)$$

using backpropagation!

When does learning take place?

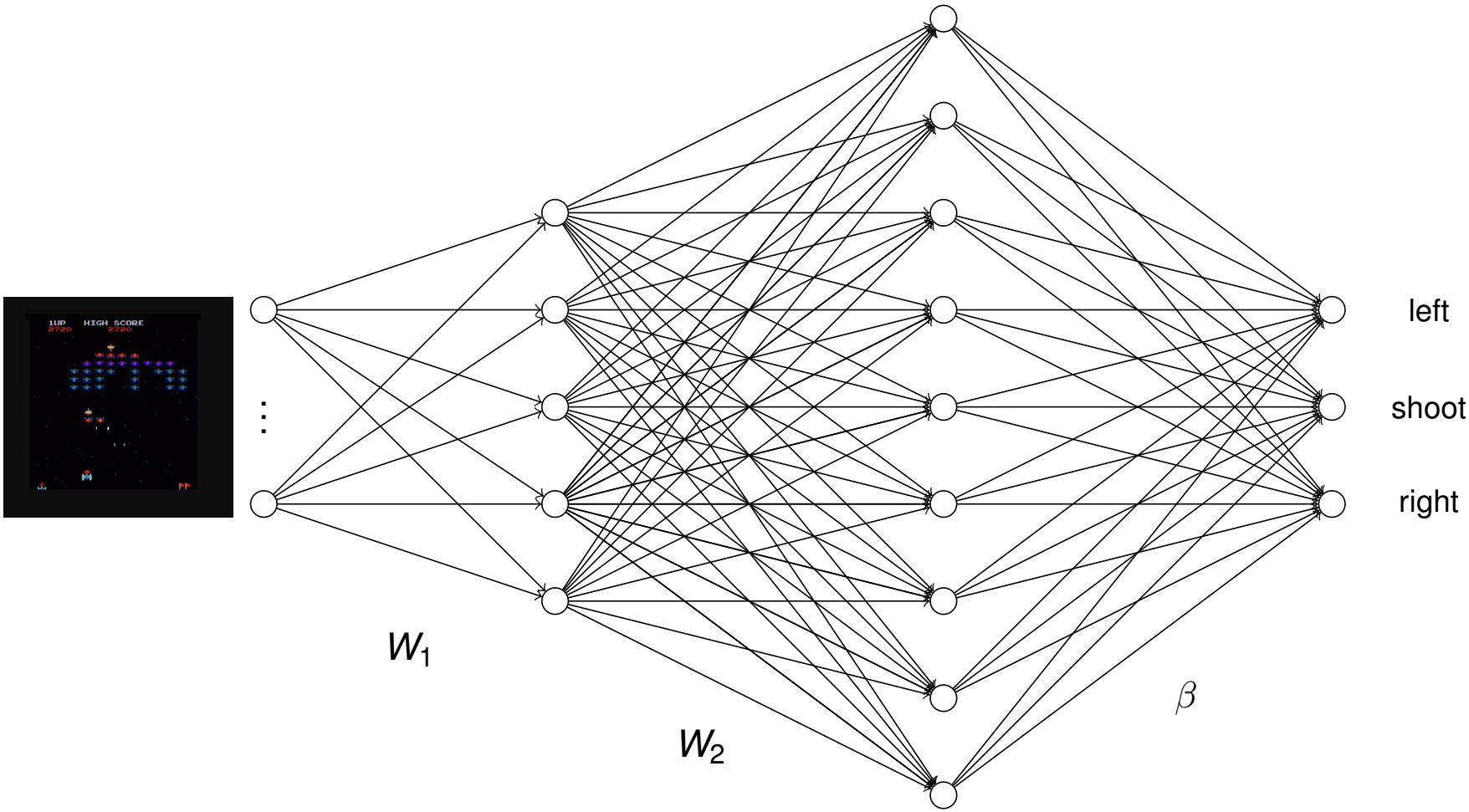
Recall from Bellman equation that y_t is an expectation.

Learning takes place when expectations are violated. The receipt of the reward itself does not cause changes.

Space Invaders



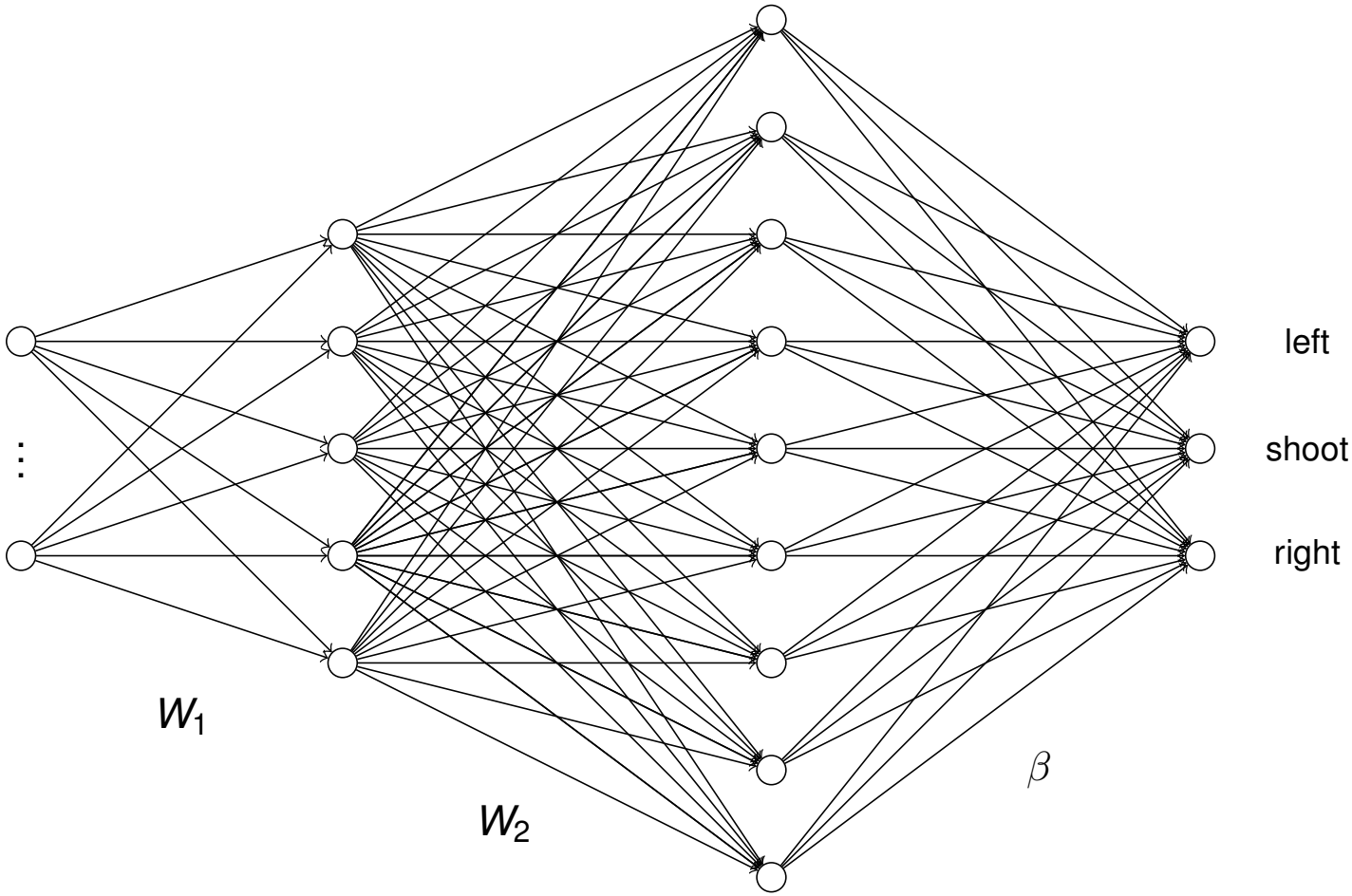
Space Invaders: Q-learning framework



Space Invaders: Q-learning framework



(multiple frames)



Early DeepMind work

- Images cropped to 84 $\times$ 84 pixels; 128 color palette; input sequence of 4 frames; reward is score
- 3-layer convolutional neural network, ReLU nonlinearity, final layer fully connected, 256 neurons
- *Q*-learning carried out over minibatches of playing sequences that are "remembered and replayed"

"Playing Atari with Deep Reinforcement Learning," Mnih et al., DeepMind Technologies, 2013.
Larger versions reached human level performance on 29/49 games.
https://patentimages.storage.googleapis.com/71/91/4a/c5cf4ffa56f705/US20150100530A1.pdf
Google acquired DeepMind and used Deep *Q*-learning to save energy (and money) in data centers.

Early DeepMind work

Role of replay buffer (inspired by neuroscience, “dreaming”)

- Prevent “forgetting” how to play early parts of a game
- Remove correlations between nearby state transitions
- Prevent cycling behavior, due to target changing

“Playing Atari with Deep Reinforcement Learning,” Mnih et al., DeepMind Technologies, 2013.
Larger versions reached human level performance on 29/49 games.
https://patentimages.storage.googleapis.com/71/91/4a/c5cf4ffa56f705/US20150100530A1.pdf
Google acquired DeepMind and used Deep *Q*-learning to save energy (and money) in data centers.

Early DeepMind work

- Identical architecture used for seven games (Beam Rider, Breakout, Enduro, Pong, Q*bert, Seaquest, Space Invaders)
- Each game trained for 100 epochs (50,000 minibatch weight updates / epoch), about 50 hours
- Surpasses human expert on three of seven games

"Playing Atari with Deep Reinforcement Learning," Mnih et al., DeepMind Technologies, 2013.
Larger versions reached human level performance on 29/49 games.
https://patentimages.storage.googleapis.com/71/91/4a/c5cf4ffa56f705/US20150100530A1.pdf
Google acquired DeepMind and used Deep *Q*-learning to save energy (and money) in data centers.

Payoff

SUSTAINABILITY

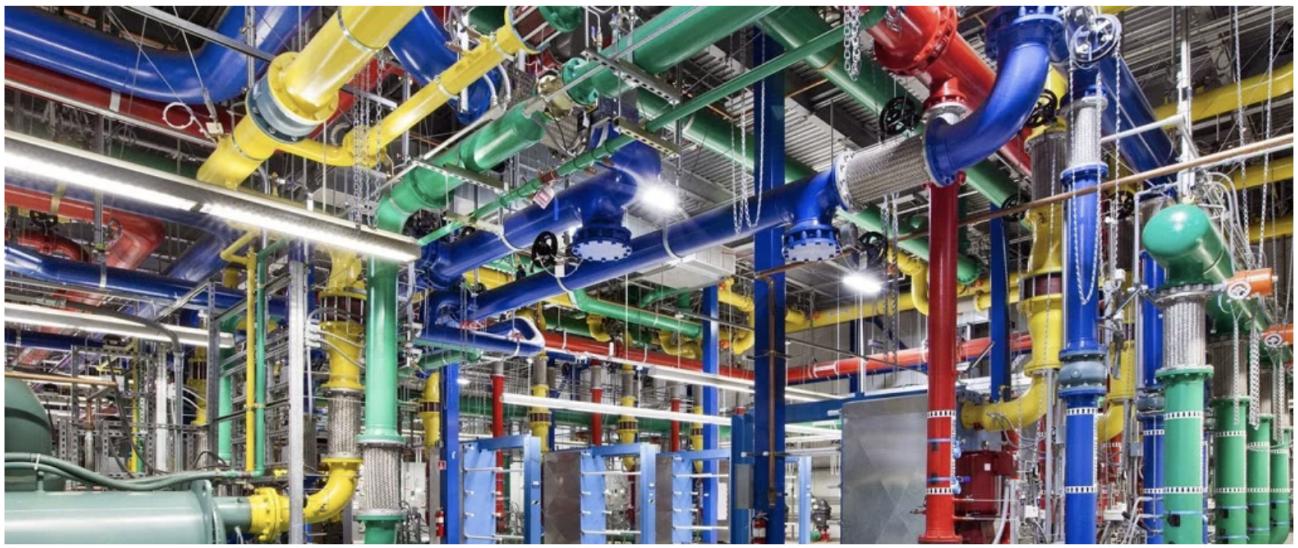
DeepMind AI reduces energy used for cooling Google data centers by 40%

Jul 20, 2016 · 4 min read

R Rich Evans
Research Engineer, DeepMind

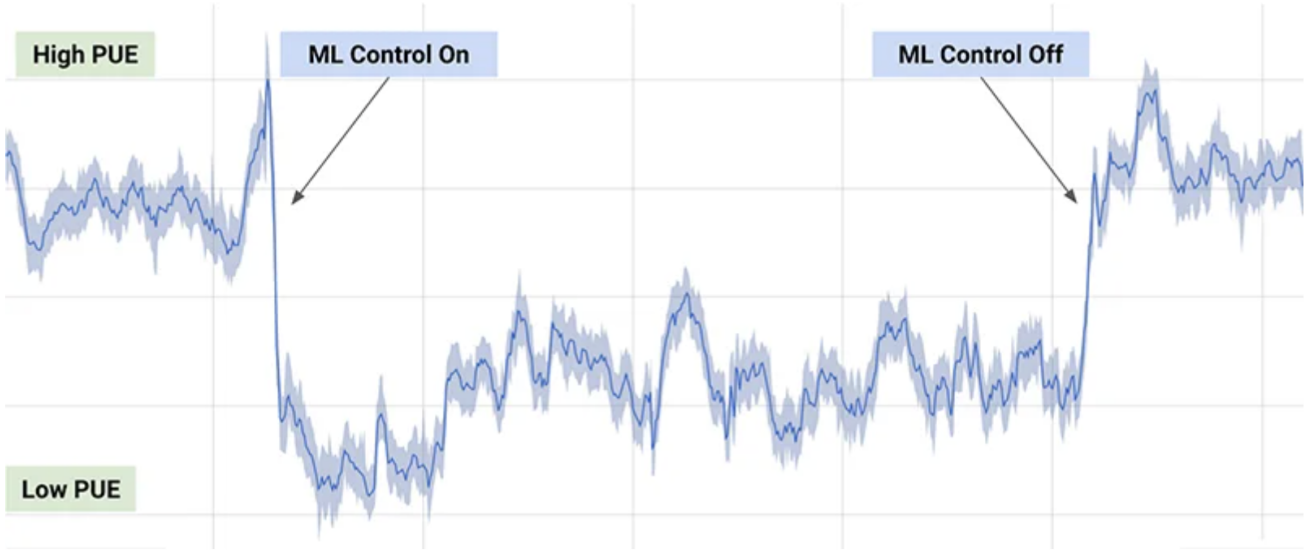
 Jim Gao
Data Center Engineer, Google

Share



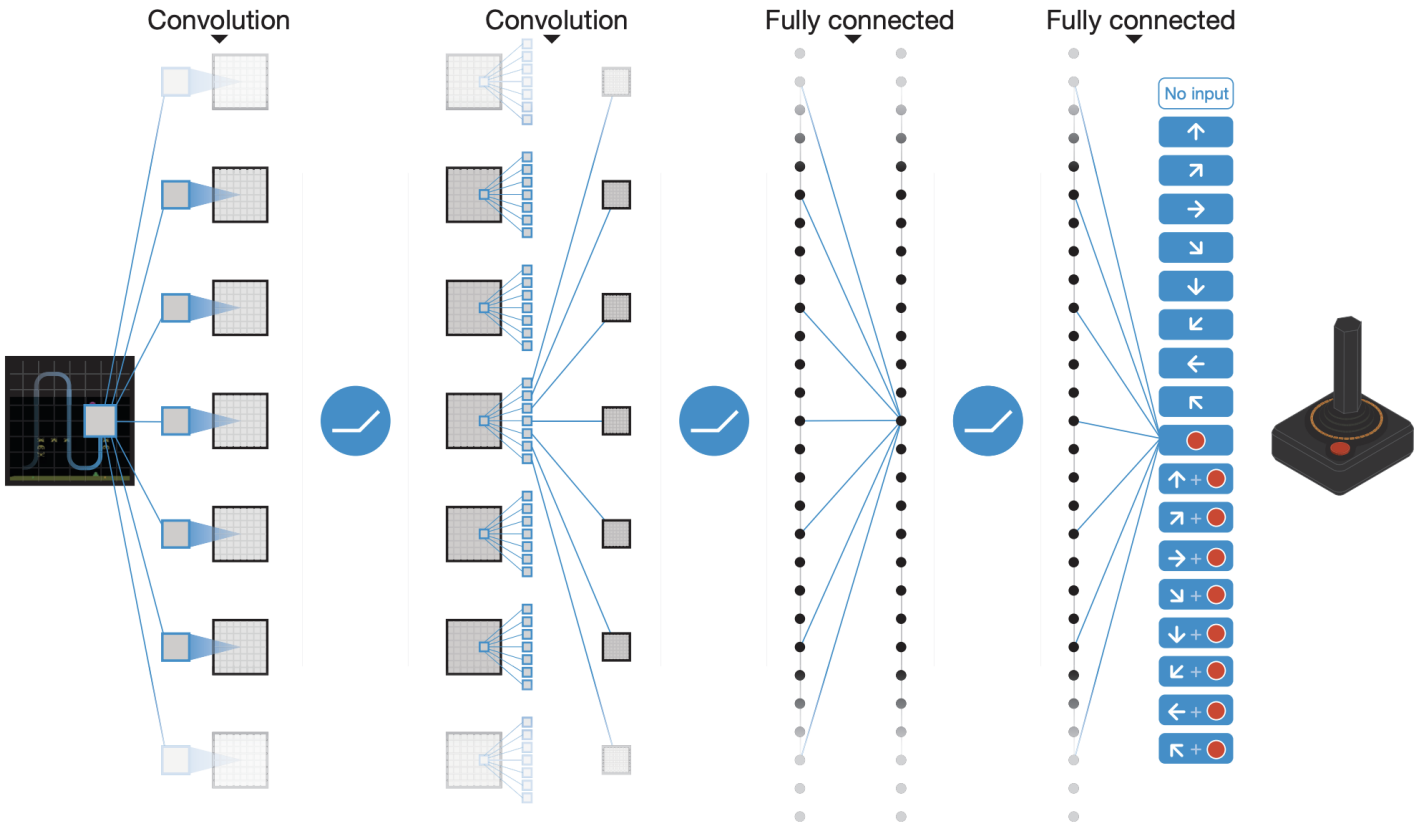
Payoff

We tested our model by deploying on a live data center. The graph below shows a typical day of testing, including when we turned the machine learning recommendations on, and when we turned them off.



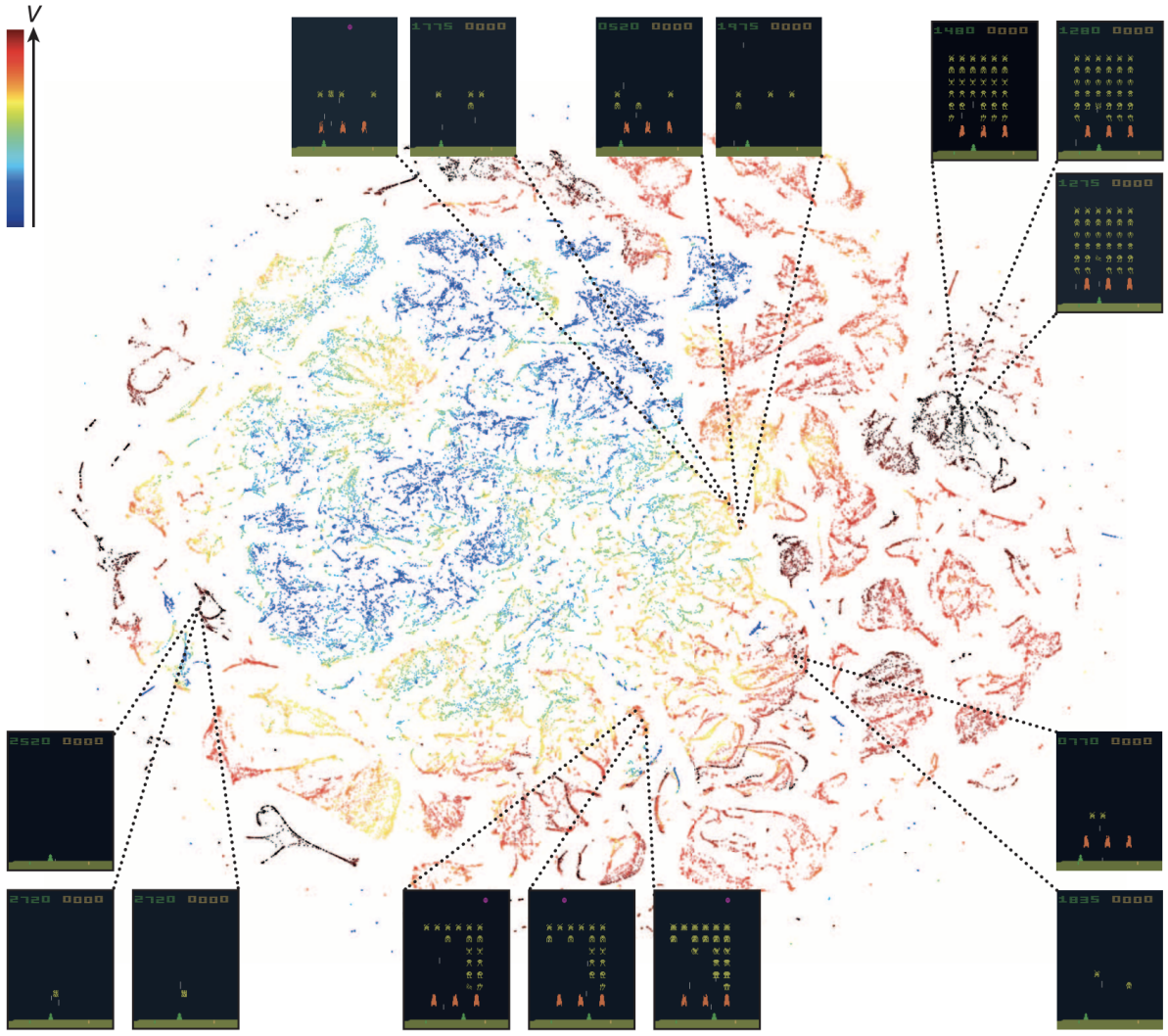
Google DeepMind graph showing results of machine learning test on power usage effectiveness in Google data centers

Second generation DQN



https://storage.googleapis.com/deepmind-data/assets/papers/DeepMindNature14236Paper.pdf

Second generation DQN: Interpretation



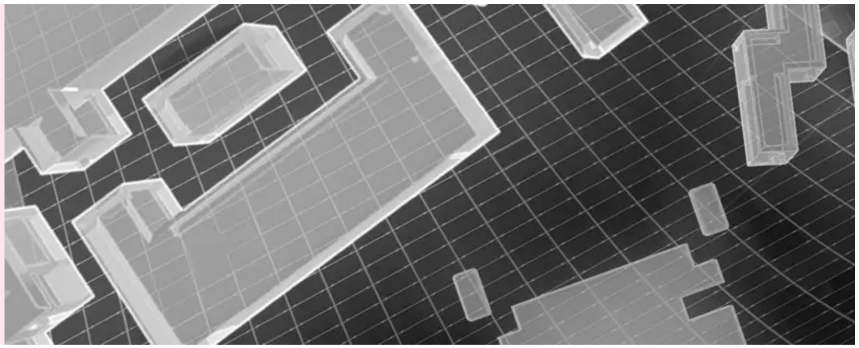
t-SNE represenations of last layer for Space Invaders, color-coded for v_*.

Further examples

BLOG POST
RESEARCH

17 JUN 2016

Deep Reinforcement Learning



SHARE

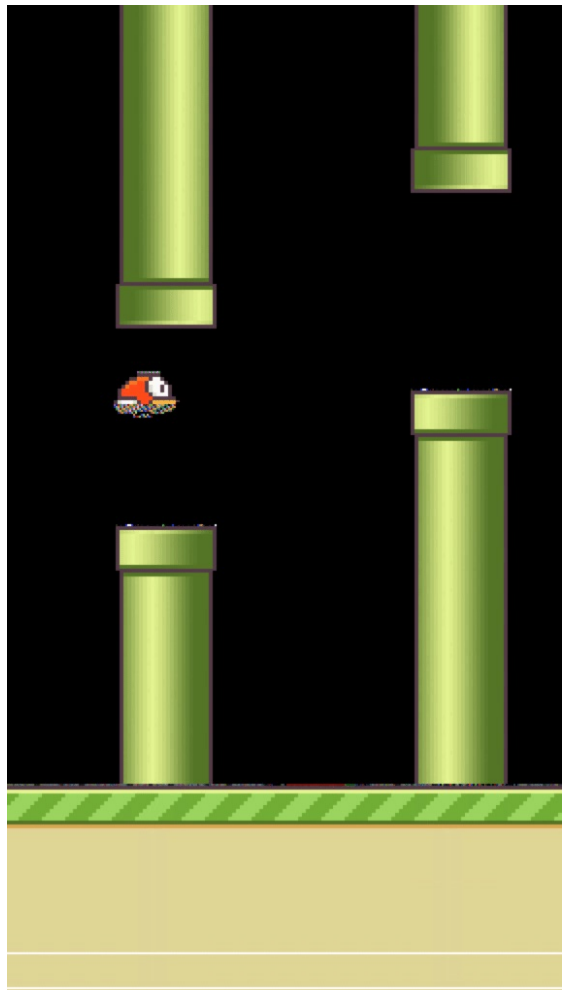
AUTHORS

DS David Silver

Humans excel at solving a wide variety of challenging problems, from low-level motor control through to high-level cognitive tasks. Our goal at DeepMind is to create artificial agents that can achieve a similar level of performance and generality. Like a human, our agents learn for themselves to achieve successful strategies that lead to the greatest long-term rewards. This paradigm of learning by trial-and-error, solely from rewards or punishments, is known as reinforcement learning (RL). Also like a human, our agents construct and learn their own knowledge directly from raw inputs, such as vision, without any hand-engineered features or domain heuristics. This is achieved by deep learning of neural

https://deepmind.com/blog/article/deep-reinforcement-learning

Assignment 4: FlapPy bird



Summary: Deep Q-learning

- For large state spaces, can't estimate Q-function directly
- Deep Q-learning: Using a (deep) neural network with state as input, value assigned to each possible action
- Success at games (Backgammon, Atari, Go)
- Close connection to neuroscience of behavior and reward

Automatic differentiation

- For supervised problems, loss function is $L(\widehat{Y}, Y)$
- Can program this directly
- RL loss functions are built up dynamically as agent makes decisions
- Automatic differentiation allows us to handle this

`https://www.tensorflow.org/guide/autodiff`

`https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html`

Automatic differentiation: Gradient collection

Modern platforms support automatic differentiation by recording relevant operations executed inside the context of a “tape”

This record is then used to compute the numerical values of gradients using “reverse mode differentiation”

---

https://en.wikipedia.org/wiki/Automatic_differentiation

Automatic differentiation: Hello world!

```
import numpy as np
import torch
```

✓ 0.5s Python

```
x = torch.tensor(3.0, requires_grad=True)

y = x**2

y.backward()
dy_dx = x.grad
print(dy_dx.numpy())
```

✓ 0.0s Python

```
6.0
```

https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html

Automatic differentiation: Hello world!

```
w = torch.randn(3, 2, requires_grad=True)
b = torch.zeros(2, dtype=torch.float32, requires_grad=True)
x = torch.tensor([[1., 2., 3.]])

y = x @ w + b
loss = torch.mean(y**2)

[dloss_dw, dloss_db] = torch.autograd.grad(loss, [w, b])
print(dloss_dw.numpy(), "\n\n", dloss_db.numpy())
```

[3] ✓ 0.0s Python

```
[[ 0.63157207 -0.3194919 ]
 [ 1.2631441  -0.6389838 ]
 [ 1.8947163  -0.9584757 ]]

 [ 0.63157207 -0.3194919 ]
```

https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html

Multi-armed bandits



Multi-armed bandits

- The rewards are independent and noisy
- Arm k has expected payoff μ_k with variance σ_k^2 on each pull
- Each time step, pull an arm and observe the resulting reward
- Played often enough, can estimate mean reward of each arm
- What is the best policy?
- Exploration-exploitation tradeoff
- “Contextual bandits” add covariate vector — but actions don’t impact the environment

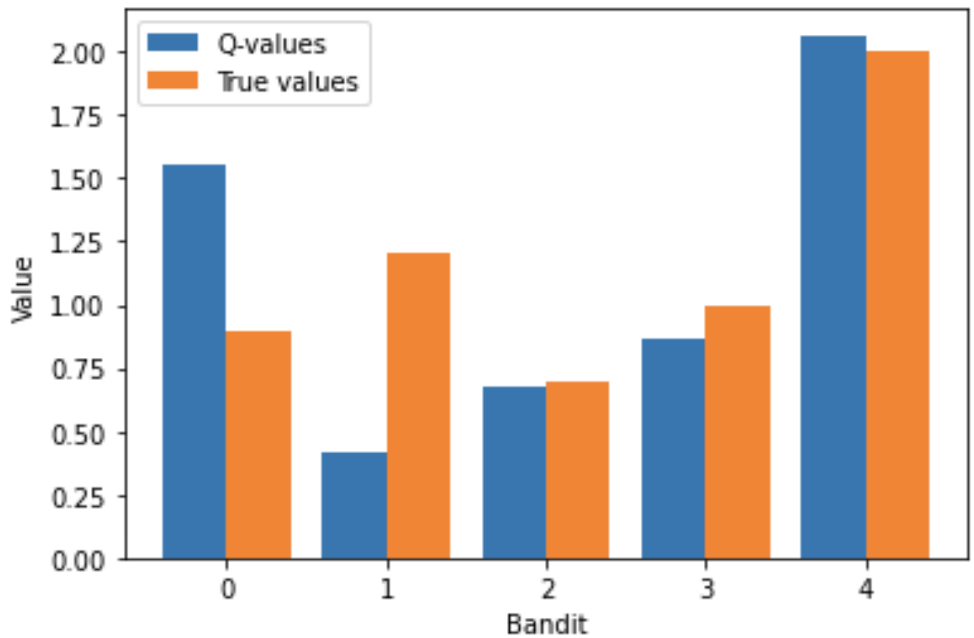
Multi-armed bandits

We'll treat this as an RL problem and hit it with a big hammer:
Deep Q-learning

Multi-armed bandits

```
======episode 10000 ======
Q-values ['1.556', '0.412', '0.675', '0.866', '2.065']
Deviation ['72.8%', '-65.7%', '-3.6%', '-13.4%', '3.3%']

<Figure size 864x504 with 0 Axes>
```

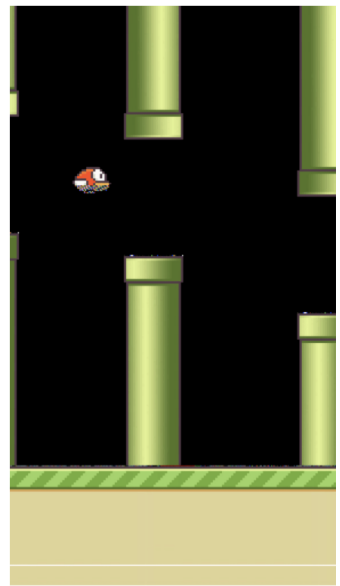


Let's go to the notebook!

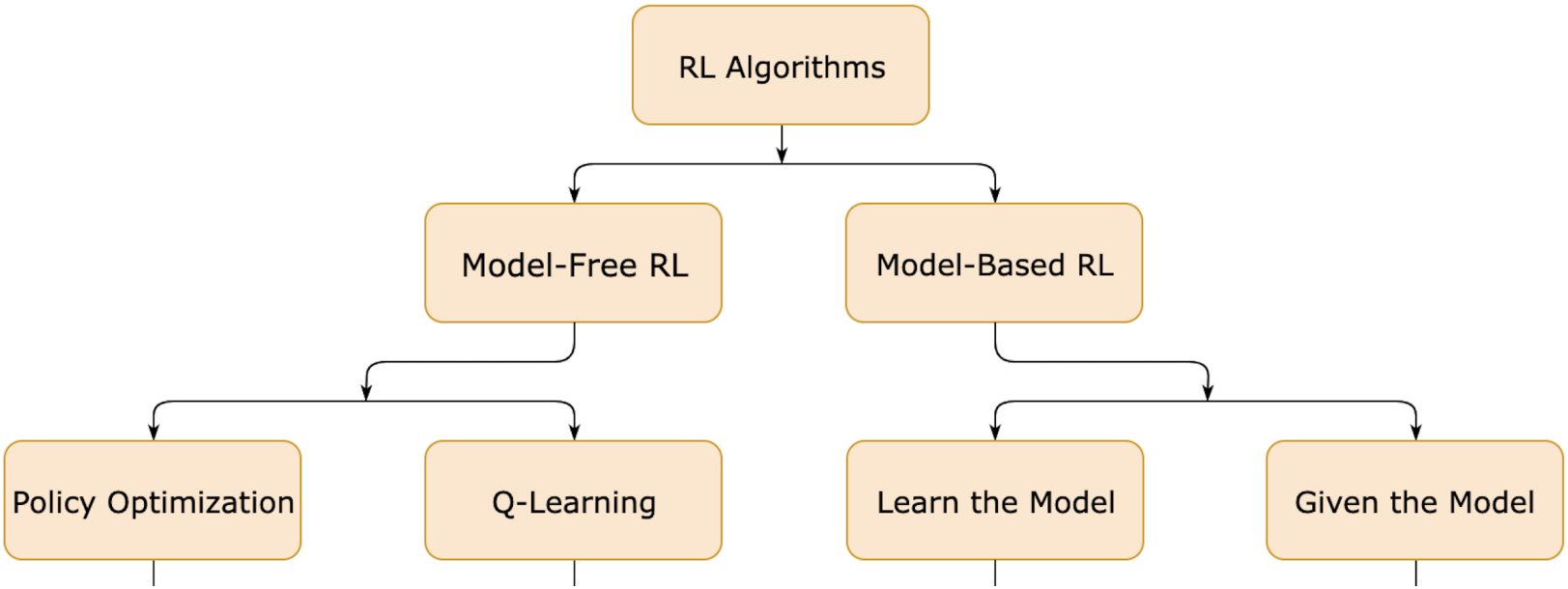
Assn 4: Flappy Bird

Problem 3: Deep Q-Learning for Flappy Bird (25 points)

In this problem, we will walk you through the implementation of deep Q learning to learn to play the Flappy Bird game.



Landscape of RL algorithms

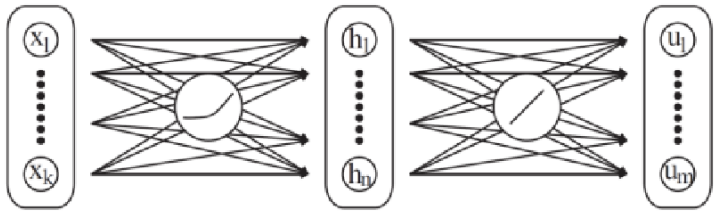


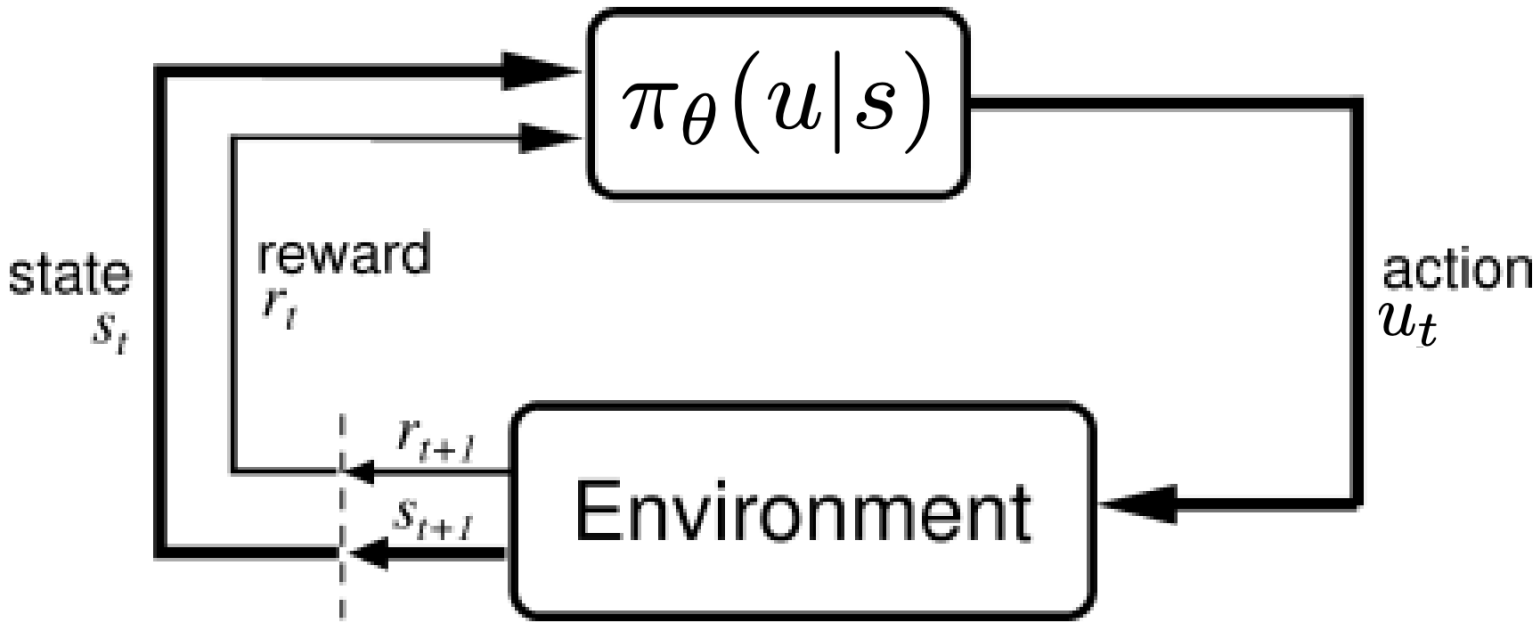
https://tinyurl.com/25vruwtp

Policy gradient methods

- policy $\pi_\theta(s)$ has parameters θ
- Perform gradient ascent over θ
- Well-suited to deep learning approaches
- Why use an on-policy method? May be possible to estimate a good policy without accurately estimating the value function or Q-function
- Can be more compute-efficient and stable than Q-learning

Policy gradient methods





Successes of policy methods



Kohl and Stone, 2004



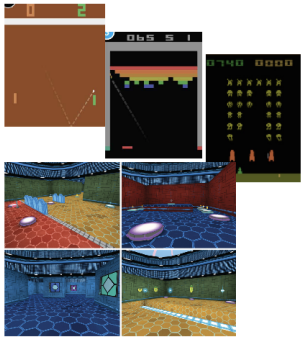
Ng et al, 2004



Tedrake et al, 2005



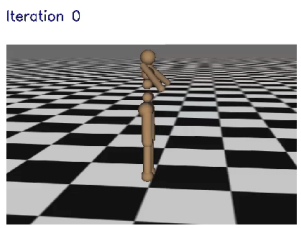
Kober and Peters, 2009



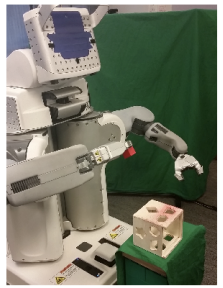
Mnih et al, 2015
(A3C)



Silver et al, 2014
(DPG)
Lillicrap et al, 2015
(DDPG)



Schulman et al,
2016 (TRPO + GAE)



Levine*, Finn*, et
al, 2016
(GPS)



Silver*, Huang*, et
al, 2016
(AlphaGo**)

John Schulman & Pieter Abbeel – OpenAI + UC Berkeley

Policy gradient methods: Loss function

We start with the loss function: Expected reward $\mathcal{J}(\theta) = \mathbb{E}(R)$

- Policy $\pi(s; \theta)$ has parameters θ
- Perform gradient ascent on $\mathcal{J}(\theta)$
- Well-suited to deep learning approaches
- Neural network policy extracts predictive "features" of states

We'll need to develop some mathematics to understand how policy methods can be efficiently applied...

Policy gradient methods: Loss function

Policy is probability distribution $\pi_\theta(a \mid s)$ over actions given state s.

The episode unfolds as a random sequence τ

$$\tau : (s_0, a_0) \to (s_1, r_1, a_1) \to (s_2, r_2, a_2) \to \cdots \to (s_T, r_T, a_T) \to s_{T+1}$$

where s_{T+1} is a terminal state. Receive reward $R(\tau)$, for example

$$R(\tau) = \sum_{t=1}^{T} r_t$$

Objective function $\mathcal{J}$ is expected reward

$$\mathcal{J}(\theta) = \mathbb{E}_\theta(R(\tau))$$

Calculating the gradient

Using Markov property, calculate $\mathbb{E}_\theta(R(\tau))$ as

$$\mathbb{E}_\theta(R(\tau)) = \int p(\tau \mid \theta) R(\tau)\, d\tau$$

$$p(\tau \mid \theta) = \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

If states or rewards are finite the integral becomes a sum, or a mix of sums and integrals.

Calculating the gradient

Using Markov property, calculate $\mathbb{E}_\theta(R(\tau))$ as

$$\mathbb{E}_\theta(R(\tau)) = \int p(\tau \mid \theta) R(\tau)\, d\tau$$

$$p(\tau \mid \theta) = \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

It follows that

$$\nabla_\theta \log p(\tau \mid \theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) = \sum_{t=0}^{T} \frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}$$

If states or rewards are finite the integral becomes a sum, or a mix of sums and integrals.

Calculating the gradient

Now we use

$$
\begin{aligned}
\nabla_\theta \mathcal{J}(\theta) &= \nabla_\theta \, \mathbb{E}_\theta R(\tau) \\
&= \nabla_\theta \int R(\tau) \; p(\tau \,|\, \theta) \; d\tau \\
&= \int R(\tau) \, \nabla_\theta p(\tau \,|\, \theta) \; d\tau \\
&= \int R(\tau) \, \frac{\nabla_\theta p(\tau \,|\, \theta)}{p(\tau \,|\, \theta)} \; p(\tau \,|\, \theta) \; d\tau \\
&= \mathbb{E}_\theta \Big(R(\tau) \nabla_\theta \log p(\tau \,|\, \theta) \Big)
\end{aligned}
$$

Need regularity conditions to exchange derivative and integral, such as uniform continuity of the derivatives of the policy.

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}
\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \nabla_\theta \log p(\tau^{(i)} \mid \theta) \\
&= \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\
&\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}
\end{aligned}$$

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}
\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \nabla_\theta \log p(\tau^{(i)} \mid \theta) \\
&= \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\
&\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}
\end{aligned}$$

$\tau^{(i)}$ is sometimes called a "rollout" of the policy π_θ

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)})\nabla_\theta \log p(\tau^{(i)} \mid \theta) \\ &= \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\ &\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}\end{aligned}$$

$\tau^{(i)}$ is sometimes called a "rollout" of the policy π_θ

The policy gradient algorithm is then

$$\theta \longleftarrow \theta + \eta \widehat{\nabla_\theta \mathcal{J}(\theta)}$$

Intuition

Gradient ascent of the expected reward with respect to policy:

- Increases probability of paths τ with large R
- Decreases probability of paths τ with small R

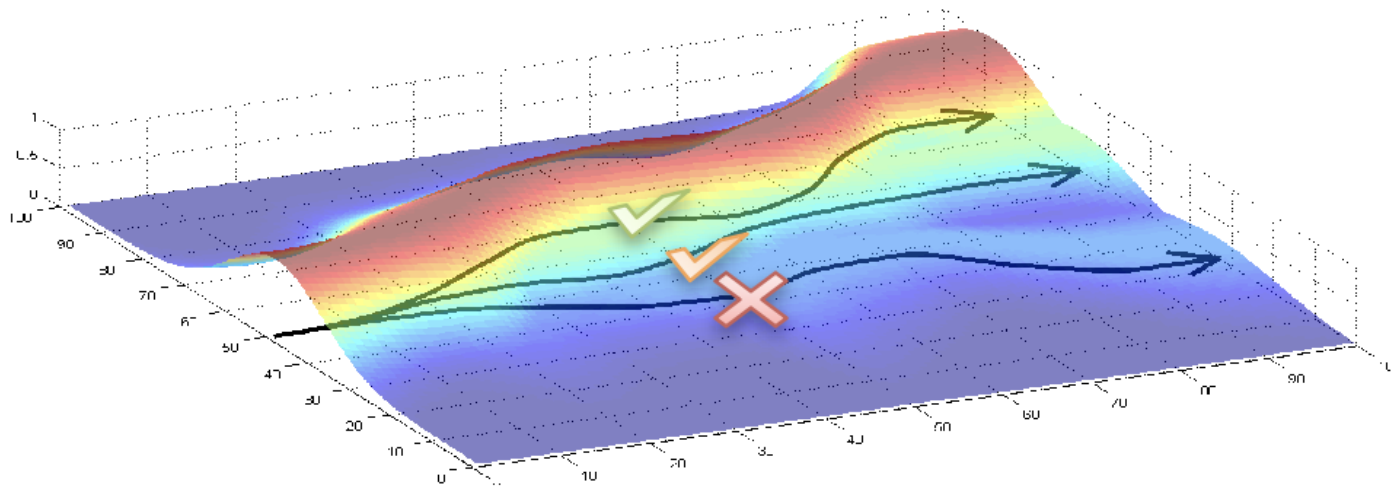
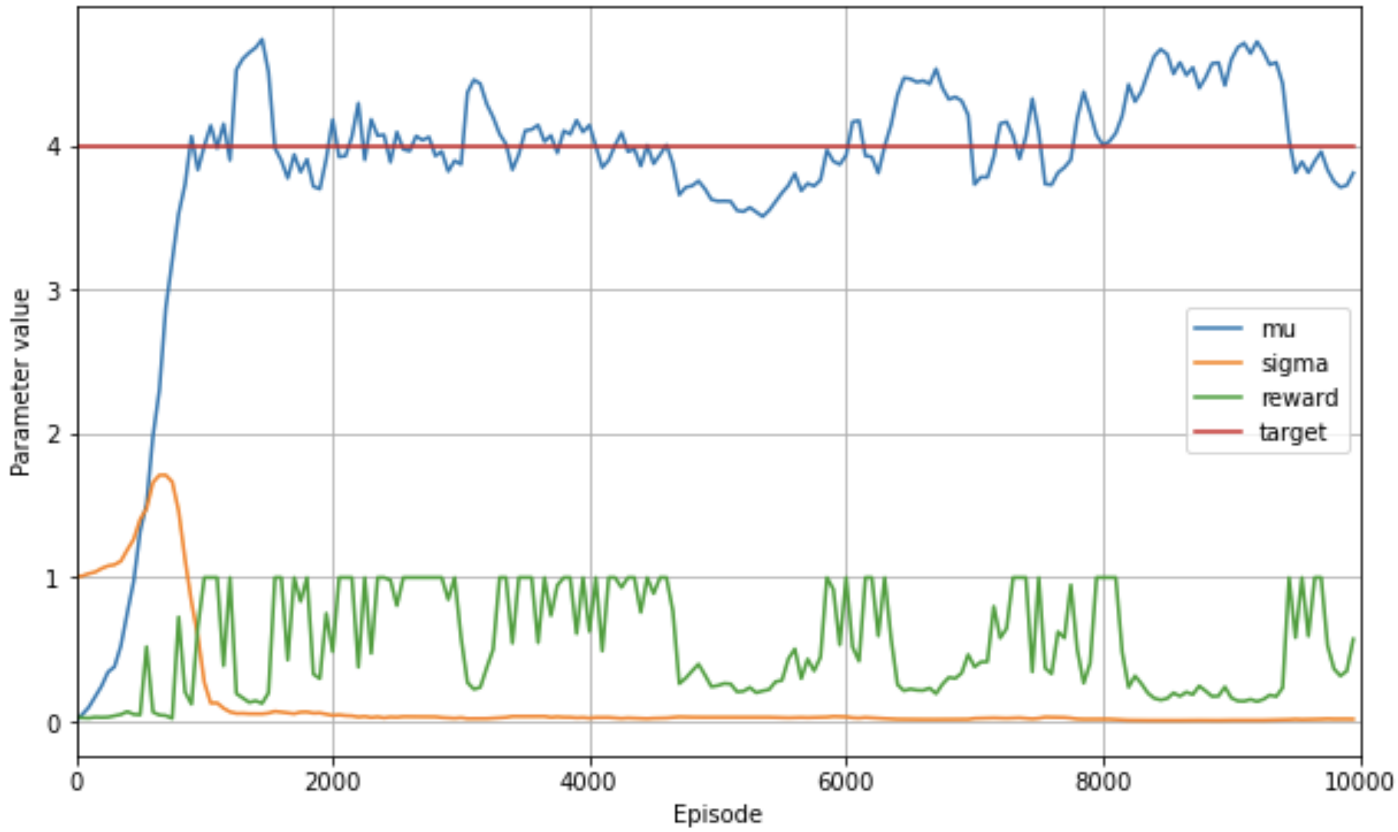


Fig from Schulman and Abbeel

Simple example



Simple example



Let’s go to the notebook

Summary: Policy gradients

- Policy methods estimate $\pi(a \mid s)$
- Tabular methods can be used for small state/action spaces
- Otherwise, parameterize $\pi_\theta(a \mid s)$ and use gradient ascent
  - Change in expected reward calculated with “grad-log trick”